

Assignment 4 – ADC differansemåling med ATmega32

Oskar Hellebust-Nygaard

9. mars 2026

Innhold

1	Innledning	2
2	Kobling	2
3	Prinsipp og kravoppfyllelse	3
4	Kodeforklaring kort	4
5	Programkode	4
6	Resultat og observasjoner	6
7	Konklusjon	6

1 Innledning

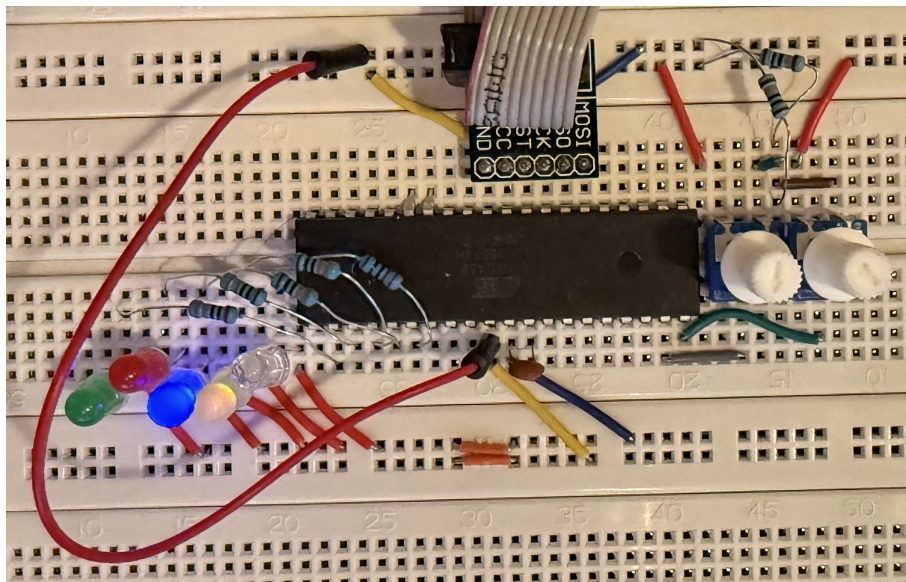
I denne oppgaven skulle vi lage et program som måler differansen mellom to analoge spenninger, $V_{in2} - V_{in1}$, med en oppløsning på 5 mV eller bedre. Basert på denne differansen skulle programmet tenne en bestemt LED, mens de andre holdes av. Hvis spenningen ligger utenfor de oppgitte intervallene, skal alle LED-ene tennes.

Kravene i oppgaven er at løsningen skal bruke **ADC Noise Reduction Mode**, **ADC auto trigger**, ta målinger hver ca. 262 ms, ha tom event-loop og bruke **switch**-setninger i stedet for **if**-setninger.

2 Kobling

Koblingen består av:

- V_{in1} kobles til ADC1 (PA1).
- V_{in2} kobles til ADC2 (PA2).
- LED0–LED4 kobles til PB0–PB4, hver med seriemotstand til GND.
- AVCC kobles til VCC.
- GND kobles til jord.
- AREF får en kondensator på 100 nF til GND.



Figur 1: Oppkobling av kretsen.

Hvorfor kondensator fra AREF til GND

Kondensatoren på AREF brukes for å gjøre referansespenningen mer stabil og mindre følsom for støy. Det gir mer stabile ADC-målinger.

3 Prinsipp og kravoppfyllelse

Direkte differansemåling

I denne løsningen brukes den differensielle ADC-funksjonen i ATmega32 direkte. Det betyr at ADC-en måler $V_{in2} - V_{in1}$ internt, i stedet for at programmet først måler to kanaler separat og deretter trekker dem fra hverandre i software. Dette gjør koden enklere og mer direkte.

Oppløsning

Vi bruker intern referanse på 2.56 V og 10-bit ADC:

$$\frac{2560 \text{ mV}}{1024} = 2.5 \text{ mV per steg}$$

Dette er bedre enn kravet om 5 mV.

Sampling hver 262 ms

For å få omtrent 262 ms mellom hver måling brukes Timer0 overflow som ADC trigger. ATmega32 kjører på 1 MHz, og Timer0 settes med prescaler 1024:

$$f_{\text{timer}} = \frac{1\,000\,000}{1024} = 976.5625 \text{ Hz}$$

Timer0 er 8-bit og teller 256 steg per overflow:

$$T = \frac{256}{976.5625} = 0.262144 \text{ s}$$

Dermed får vi:

$$T = 262.144 \text{ ms}$$

Dette oppfyller kravet.

ADC Noise Reduction Mode

Prosessoren settes i ADC Noise Reduction sleep mode. Da stopper CPU-en mens ADC-målingen skjer, og det reduserer digital støy som kan påvirke målingen.

Tom event-loop

Hovedløkka inneholder ingenting. Programmet sover i løkka og våkner bare når et interrupt oppstår.

Switch i stedet for if

Både valg av spenningsområde og valg av LED gjøres med `switch`.

Clearing av timer-flag

Når ADC auto trigger brukes sammen med timer, må trigger-flagget cleares ved å skrive logisk 1 til flagget. Dette gjøres i timer-interruptet.

4 Kodeforklaring kort

Programmet gjør dette:

1. Setter PB0–PB4 som utganger til LED-er.
2. Setter opp Timer0 slik at overflow skjer hver 262.144 ms.
3. Setter opp ADC til differensiell måling $ADC2 - ADC1$ med intern 2.56 V referanse.
4. Lar Timer0 overflow trigge ADC automatisk.
5. Når ADC er ferdig, leses resultatet, konverteres til signert verdi og deretter til millivolt.
6. En switch-setning velger hvilken LED som skal tennes.
7. Hovedprogrammet sover hele tiden i ADC Noise Reduction Mode.

5 Programkode

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <stdint.h>

#define LED0 PB0
#define LED1 PB1
#define LED2 PB2
#define LED3 PB3
#define LED4 PB4

static void show_leds(uint8_t mask)
{
    PORTB = (PORTB & ~0x1F) | (mask & 0x1F);
}

ISR(TIMER0_OVF_vect)
{
    // clear Timer0 overflow flag
    TIFR |= (1 << TOV0);
}

ISR(ADC_vect)
{
    int16_t adc_value;
    int16_t vin_mV;
    uint8_t zone;

    // les ADC-resultatet
    adc_value = ADC;
```

```

// sign-extend 10-bit differensiell verdi til 16-bit
if (adc_value & 0x0200)
{
    adc_value |= 0xFC00;
}

// konverter til millivolt
vin_mV = (int32_t)adc_value * 2560 / 1024;

// anbefalt ved differensiell måling etter sleep
ADCSRA &= ~(1 << ADEN);
ADCSRA |= (1 << ADEN);

// utenfor intervallet -> alle på
switch ((vin_mV < -2500) || (vin_mV > 2500))
{
    case 1:
        zone = 5;
        break;

    default:
        // flytt området [-2500,2500] til [0,5000]
        switch ((vin_mV + 2500) / 1000)
        {
            case 0: zone = 0; break; // -2500 to -1501
            case 1: zone = 1; break; // -1500 to -501
            case 2: zone = 2; break; // -500 to 499
            case 3: zone = 3; break; // 500 to 1499
            case 4: zone = 4; break; // 1500 to 2499
            case 5: zone = 4; break; // 2500
            default: zone = 5; break;
        }
        break;
}

switch (zone)
{
    case 0: show_leds(1 << LED0); break;
    case 1: show_leds(1 << LED1); break;
    case 2: show_leds(1 << LED2); break;
    case 3: show_leds(1 << LED3); break;
    case 4: show_leds(1 << LED4); break;
    default: show_leds(0x1F); break; // alle på
}
}

int main(void)
{
    // PB0-PB4 som output
    DDRB |= 0x1F;

```

```

PORTB &= ~0x1F;

// ADC2 - ADC1, intern 2.56 V referanse, høyrejustert
ADMUX = (1 << REFS1) | (1 << REFS0) | 0b10010;

// ADC enable, interrupt, auto trigger, prescaler 8
ADCSRA = (1 << ADEN) |
          (1 << ADIE) |
          (1 << ADSCF) |
          (1 << ADPS1) |
          (1 << ADPS0);

// Auto trigger source = Timer0 Overflow
SFIOR = (1 << ADTS2);

// Timer0 normal mode, prescaler 1024
TCCR0 = (1 << CS02) | (1 << CS00);

// enable Timer0 overflow interrupt
TIMSK |= (1 << TOIE0);

// ADC Noise Reduction Mode
set_sleep_mode(SLEEP_MODE_ADC);

sei();

while (1)
{
    sleep_mode();
}
}

```

6 Resultat og observasjoner

Programmet måler differansen direkte som $V_{in2} - V_{in1}$ og tenner riktig LED avhengig av hvilket intervall spenningen ligger i. Hvis verdien er mindre enn -2500 mV eller større enn 2500 mV, tennes alle LED-ene. Det tas automatisk nye målinger omtrent hver 262 ms.

Løsningen er enklere enn å måle to kanaler annenhver gang, fordi vi slipper ekstra variabler for fase og mellomlagring av to målinger. Koden blir derfor kortere og lettere å forstå.

7 Konklusjon

Løsningen oppfylles kravene i Assignment 4 ved å bruke ADC Noise Reduction Mode, ADC auto trigger, sampling hver ca. 262 ms, tom event-loop og **switch**-setninger. Ved å bruke differensiell ADC direkte blir løsningen enklere enn en variant der V_{in1} og V_{in2} måles separat og trekkes fra hverandre etterpå.